
COMS W4167: Computer Animation

Programming Assignment 5

Due 9:00PM Sun. April 26th, 2026 (EST)

Academic Honesty Policy

You are permitted and encouraged to discuss your work with other students. Except where explicitly stated otherwise, you may work out equations in writing on paper or a whiteboard. You are encouraged to use **Ed Discussion** to converse with other students, the TAs, and the instructor.

HOWEVER, you must NOT share source code or hard copies of source code. Refrain from activities or the sharing materials that could cause your source code to **APPEAR TO BE** similar to another student's enrolled in this or previous years. We will be monitoring source code for individuality. Cheating will be dealt with severely. Source code should be yours and yours only. Do not cheat. For more details, please refer to the full academic honesty policy on the Engineering School's website.

Chapter 1

Course Notes

1.1 Continuum Mechanics

1.1.1 Kinematics

The kinematics of a deforming material are represented by a *deformation mapping* $\Phi : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ taking every ‘undeformed’ material point $\mathbf{x}$ on the plane to its corresponding deformed position $\Phi(\mathbf{x})$. In general, Φ is an arbitrary nonlinear map taking 3 coordinates denoting a position to a new set of 3 coordinates denoting some other position.

We’d like to know how much this deformation is stretching, compressing or shearing the material. Since a generic deformation is an arbitrary nonlinear map, we only look at each point locally (by examining its infinitesimal neighborhood). Consider a point $\bar{\mathbf{x}}$ in the undeformed space, and a nearby point $\bar{\mathbf{x}} + \delta\bar{\mathbf{x}}$. The deformed position are $\mathbf{x}$ and $\mathbf{x} + \delta\mathbf{x}$, and by definition of Φ we have:

$$\begin{aligned}\mathbf{x} &= \Phi(\bar{\mathbf{x}}) \\ \mathbf{x} + \delta\mathbf{x} &= \Phi(\bar{\mathbf{x}} + \delta\bar{\mathbf{x}}) \\ &= \Phi(\bar{\mathbf{x}}) + \nabla\Phi(\bar{\mathbf{x}})\delta\bar{\mathbf{x}} + O(\|\delta\bar{\mathbf{x}}\|^2) \\ \delta\mathbf{x} &= \nabla\Phi(\bar{\mathbf{x}})\delta\bar{\mathbf{x}} + O(\|\delta\bar{\mathbf{x}}\|^2)\end{aligned}$$

$\nabla\Phi(\bar{\mathbf{x}})$ is the Jacobian of the transformation Φ evaluated at $\bar{\mathbf{x}}$, often called the *Deformation Gradient*. The high order terms in the end can be omitted because we assume $\delta\bar{\mathbf{x}}$ is infinitesimal.

1.1.2 The Strain

Now we are ready to evaluate how much lengths change after such a transformation. Under the assumption that $\delta\bar{\mathbf{x}}$ is infinitesimal, the line from $\bar{\mathbf{x}}$ to $\bar{\mathbf{x}} + \delta\bar{\mathbf{x}}$ in the undeformed space remains a line after the deformation, going from $\mathbf{x}$ to $\mathbf{x} + \delta\mathbf{x}$. Therefore the lengths of this piece of material before and after deformation are $\|\delta\bar{\mathbf{x}}\|$ and $\|\delta\mathbf{x}\|$ respectively. For the ease of computation we look at the change in squared length (so as to avoid taking square roots):

$$\begin{aligned}\delta\mathbf{x}^T\delta\mathbf{x} - \delta\bar{\mathbf{x}}^T\delta\bar{\mathbf{x}} &= (\nabla\Phi(\bar{\mathbf{x}})\delta\bar{\mathbf{x}})^T(\nabla\Phi(\bar{\mathbf{x}})\delta\bar{\mathbf{x}}) - \delta\bar{\mathbf{x}}^T\delta\bar{\mathbf{x}} \\ &= \delta\bar{\mathbf{x}}^T\nabla\Phi(\bar{\mathbf{x}})^T\nabla\Phi(\bar{\mathbf{x}})\delta\bar{\mathbf{x}} - \delta\bar{\mathbf{x}}^T\delta\bar{\mathbf{x}} \\ &= \delta\bar{\mathbf{x}}^T[\nabla\Phi(\bar{\mathbf{x}})^T\nabla\Phi(\bar{\mathbf{x}}) - I]\delta\bar{\mathbf{x}}\end{aligned}$$

The reason we can use change in squared length instead of change in length itself as a measure of stretching is that, to the first order, they are equivalent as long as the deformation is small. In order to make this

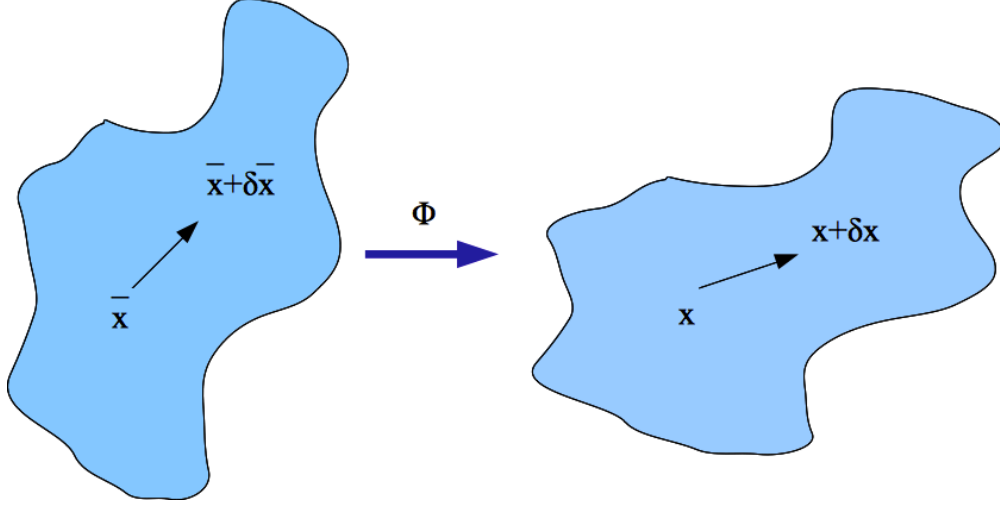


Figure 1.1: The deformation map.

change in squared length a scale-invariant representation of relative stretching/compression, we extract out the term in the middle, $[\nabla\Phi(\bar{\mathbf{x}})^T \nabla\Phi(\bar{\mathbf{x}}) - I]$, and call it the *strain*, or ϵ .

This strain is a second-order tensor, unlike the 1D case where a single scalar fully describes how much a spring is stretched as shown in class two weeks ago. In 3D, materials can be stretched differently in different directions, so a second-order tensor is needed. The physical meaning of this tensor is that when you hit it on both sides with a direction $\delta\bar{\mathbf{x}}$, it returns a change in squared length in that direction. In its matrix representation, the eigenvectors of this matrix are the two directions that are stretched or compressed the most, and the corresponding eigenvalues tell you how much the stretching/compression is in that direction.

Another interesting thing to note is that ϵ is rotation-invariant, even though $\nabla\Phi$ is not. As a measure of deformation, the strain should not depend on any rigid motion such as translation and rotation. It is obvious that $\nabla\Phi$, which is the result of a spatial differential operator, stays the same under translation, but what about rotation? To test this, we apply a constant rotation Q to any deformation Φ to obtain a new deformation $\tilde{\Phi}$, and see how ϵ responds:

$$\begin{aligned}
 \tilde{\Phi} &= Q\Phi \\
 \nabla\tilde{\Phi} &= Q\nabla\Phi \\
 \tilde{\epsilon} &= \nabla\tilde{\Phi}^T \nabla\tilde{\Phi} - I \\
 &= (Q\nabla\Phi)^T (Q\nabla\Phi) - I \\
 &= \nabla\Phi^T Q^T Q \nabla\Phi - I \\
 &= \nabla\Phi^T \nabla\Phi - I \\
 &= \epsilon
 \end{aligned}$$

Thus we've convinced ourselves that our strain ϵ is invariant under any rigid motions, therefore it qualifies as a measure of deformation.

1.1.3 Example Deformation (2D)

Let's look at a 2D example with real numbers. Consideration an affine deformation of a rectangular sheet of rubber as in Figure 1.2. This rectangle is stretched along the green axis by 80%, and compressed along the red axis by 40%. This means our deformation map Φ is multiplication by a matrix with two eigenvalues:

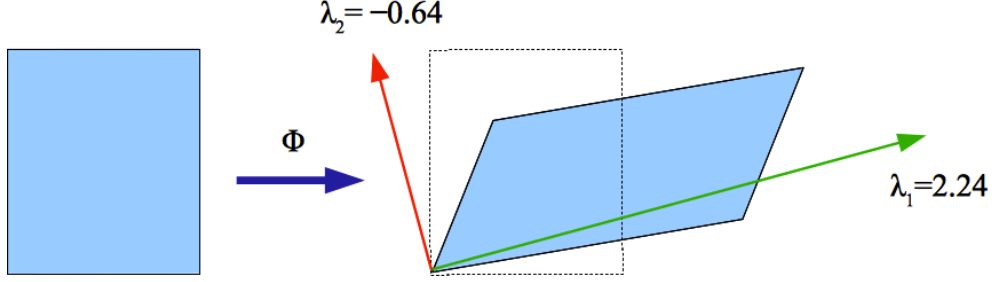


Figure 1.2: An example deformation of square, with stretching in one direction and compression in the other. The two arrows denote the two eigenvectors of the strain tensor with corresponding eigenvalues (λ_1 and λ_2) marked on the arrows.

1.8 corresponding to the eigenvector (3, 1) (green axis), and 0.6 corresponding to the eigenvector (-1, 3) (red axis). Therefore Φ looks like this:

$$\begin{aligned}\Phi(\mathbf{x}) &= \frac{1}{\sqrt{10}} \begin{pmatrix} 3 & -1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 1.8 & 0 \\ 0 & 0.6 \end{pmatrix} \frac{1}{\sqrt{10}} \begin{pmatrix} 3 & 1 \\ -1 & 3 \end{pmatrix} \mathbf{x} \\ &= \begin{pmatrix} 1.68 & 0.36 \\ 0.36 & 0.72 \end{pmatrix} \mathbf{x}\end{aligned}$$

We can compute the strain tensor ϵ from Φ :

$$\begin{aligned}\epsilon &= \nabla\Phi(\bar{\mathbf{x}})^T \nabla\Phi(\bar{\mathbf{x}}) - I \\ &= \begin{pmatrix} 1.68 & 0.36 \\ 0.36 & 0.72 \end{pmatrix}^T \begin{pmatrix} 1.68 & 0.36 \\ 0.36 & 0.72 \end{pmatrix} - I \\ &= \begin{pmatrix} 1.952 & 0.864 \\ 0.864 & 0.352 \end{pmatrix}\end{aligned}$$

It is easy to verify that this 2x2 matrix has eigenvalues 2.24 and -0.64, corresponding to a 80% stretching ($1.8^2 - 1 = 2.24$) and a 40% compression ($0.6^2 - 1 = -0.64$).

1.2 Physics

The strain tensor gives us a complete description of the deformation of the material, but it doesn't tell us how the material will respond. That is out of the scope of kinematics, and we have to put in some ingredients from physics. Here in this section we'll attempt to write down the energy that arises from the deformation, which governs how the simulation will take place in time. For this purpose, we can proceed in either a mechanics-oriented way, or a mathematics-oriented way.

1.2.1 The Mathematical Way

Imagine we have somehow obtained the energy function w of strain ϵ , we can take its Taylor expansion:

$$w(\epsilon) = w(0) + \frac{\partial w(0)}{\partial \epsilon} \epsilon + \frac{1}{2} \frac{\partial^2 w(0)}{\partial \epsilon^2} \epsilon^2 + O(\|\epsilon\|^3), \dots,$$

where $w(0)$ is a constant independent of ϵ and can be conveniently assume to be 0. The linear term is also 0 because the energy should be minimized at zero strain (the definition of “resting shape”). If we adopt

the *small strain assumption*, the higher order terms can be omitted since the energy is dominated by the quadratic term. This is basically saying, no matter how complicated the material laws are, under the small strain assumption we can always write down the energy as a quadratic function of the strain.

1.2.2 The Mechanical Way

Physically, the energy is equal to the product between the stress and the strain:

$$w(\epsilon) = \frac{1}{2} \sum_{i=1}^3 \sum_{j=1}^3 \epsilon_{ij} \sigma_{ij}$$

and the stress tensor σ is defined as the elastic moduli times the strain:

$$\sigma_{kl} = \sum_{i=1}^3 \sum_{j=1}^3 c_{ijkl} \epsilon_{ij}$$

Therefore, the energy is:

$$w(\epsilon) = \frac{1}{2} \sum_{i=1}^3 \sum_{j=1}^3 \sum_{k=1}^3 \sum_{l=1}^3 c_{ijkl} \epsilon_{ij} \epsilon_{kl}$$

Here we obtain an energy that is quadratic in strain again. c is a fourth-order tensor that encodes all the material properties such as stiffness upon stretching/compression in each direction and shearing etc. Although a generic fourth-order tensor requires a 81 scalars to represent, symmetry in both the strain tensor and the stress tensor reduces this number to 21. If we further assumes that the material is isotropic, then we only need two parameters to fully describe this material: *Young's modulus* E and *Poisson ratio* ν :

$$\begin{pmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \\ \sigma_{xz} \\ \sigma_{yz} \end{pmatrix} = \frac{E}{(1+\nu)(1-2\nu)} \begin{pmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{pmatrix} \begin{pmatrix} \epsilon_{xx} \\ \epsilon_{yy} \\ \epsilon_{zz} \\ 2\epsilon_{xy} \\ 2\epsilon_{xz} \\ 2\epsilon_{yz} \end{pmatrix}$$

1.2.3 Total Energy

The $w(\epsilon)$ defined above gives the energy density at a point in the material. Since ϵ is a function of undeformed space position $\bar{x}$, so is w . To obtain the total elastic energy that is stored in an elastic body, we integrate this energy density over the entire body:

$$W = \int_{\Omega} w(\bar{x}) d\Omega \quad (1.1)$$

In practice, we often assume that the strain is constant for any element (discussed in the next section), therefore the energy is simplified to $W = w(\bar{x})A$ where A is the 2D element's area or the 3D element's volume.

1.2.4 Discretization

All the analysis up to now has been performed on a continuum, in the form of differential equations. Continua have an infinite number of degrees of freedom and cannot be manipulated directly in computers—we need to *discretize*.

The discretization here is in the spatial dimensions, but it's in the same spirit with the temporal discretization we've implemented in PA1 and PA2. By spatial discretization we represent the body of interest with a set of nodes, connected by edges, polygon faces or polyhedron volumes, depending on the dimension of the body. These building blocks are called *elements*. There are a fixed number of nodes in each element, and due to the limited amount of information these nodes can provide, we have to assume certain properties on the element itself. For example, a triangle element with only one node in each corner (one of the simplest and most commonly used element types) simply does not have any information about how the deformation mapping changes over the triangle to the second or higher order, so we have to assume that the deformation Φ is linear, and when we need information about a point inside the triangle, we linearly interpolate between the three nodes. Linear deformation map implies constant deformation gradient, and as a result this element type is called *constant strain triangle*.

Of course the interpolation won't be able to faithfully reproduce the true property values at an arbitrary point in the element, so a *discretization error* is introduced; but since our hardware technology isn't advanced enough to handle infinite degrees of freedom, this error is inevitable. As long as the discretization error approaches zero as we refine our mesh further and further, we say the discretization *converges* and thus is valid. For our milestone, we do not consider more complicated (but potentially more accurate) elements.

1.3 Cloth Simulation

In Sec. 1.1, we have derived the energy function $w(\epsilon)$ for a continuous elastic body. At the very bottom level, the energy function is defined on a single element based on the positions of the element's nodes. If we apply this idea to simulate a cloth, the thin thickness of the cloth demands an excessive number of elements to represent the cloth, and render the cloth computationally intractable.

The derivation in Sec. 1.1 inspires us to design energy functions for cloth as a 2D surface mesh, not a 3D volumetric mesh. In this way, the mesh resolution will not be restricted by the cloth's thickness, and we have the freedom to "design" different energy functions for cloth deformation, including stretch, shear, and bending. These energy functions may not be physically *principled*, but rather physically *inspired*. When designed properly, they can produce realistic cloth simulation results.

In this section, we follow the seminar paper on cloth simulation [1]. **You will implement a cloth simulator based on this paper with some simplifications.** I urge you to read the paper to understand all details.

1.3.1 Energies and Forces

The energy function is defined on a single triangle for resisting stretch and shear and on a single edge for resisting bending around that edge. For each segment, the energy function is defined as

$$E = \frac{1}{2}kC(\mathbf{x})^T C(\mathbf{x}), \quad (1.2)$$

where k is the stiffness of that energy function, C measures the constraint violation, and $\mathbf{x}$ is the vertex positions of that element.

In what follows, I'll derive the detailed expressions for stretch, shear, and bending forces. Again, **please refer to the paper [1] for detailed explanations of these forces.** Here I focus mainly on the derivations needed to implement the assignment (which are not given in the paper).

Stretch Force

Each cloth vertex has a 2D coordinate in its undeformed space. Let $\mathbf{u}_i$ be the 2D coordinate of vertex i . In the assignment code, the 2D coordinate is stored in the `particle_uv` field of the `Model` class. For each triangle with three vertices $\mathbf{x}_0$, $\mathbf{x}_1$, and $\mathbf{x}_2$, the 2D coordinates are $\mathbf{u}_0$, $\mathbf{u}_1$, and $\mathbf{u}_2$ respectively. Define $\Delta\mathbf{u}_1 = \mathbf{u}_1 - \mathbf{u}_0$

and $\Delta \mathbf{u}_2 = \mathbf{u}_2 - \mathbf{u}_0$.

$$\begin{bmatrix} \mathbf{w}_u & \mathbf{w}_v \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 \end{bmatrix} \begin{bmatrix} \Delta \mathbf{u}_1 & \Delta \mathbf{u}_2 \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 \end{bmatrix} \begin{bmatrix} d_{00} & d_{01} \\ d_{10} & d_{11} \end{bmatrix} \quad (1.3)$$

This is the Eq. (9) in the paper [1]. Thus, $\mathbf{w}_u$ and $\mathbf{w}_v$ can be expressed as

$$\begin{aligned} \mathbf{w}_u &= d_{00}(\mathbf{x}_1 - \mathbf{x}_0) + d_{10}(\mathbf{x}_2 - \mathbf{x}_0) \\ \mathbf{w}_v &= d_{01}(\mathbf{x}_1 - \mathbf{x}_0) + d_{11}(\mathbf{x}_2 - \mathbf{x}_0). \end{aligned} \quad (1.4)$$

The constraints for stretch can be written as

$$C = \begin{bmatrix} C_u \\ C_v \end{bmatrix} = \begin{pmatrix} \|\mathbf{w}_u\|_2 - b_u \\ \|\mathbf{w}_v\|_2 - b_v \end{pmatrix}, \quad (1.5)$$

We use $b_u = 1$ and $b_v = 1$ in the assignment code. The stretch energy for the considered triangle is

$$E_s = \frac{1}{2} a k_s (C_u^2 + C_v^2), \quad (1.6)$$

where a is the triangle area. Note that the C function here is slightly different from (10) in the paper [1]: we don't include the triangle area a here, but include it in (1.6), because the energy must be scaled by the surface area.

The forces on $\mathbf{x}_i$ caused by stretching along u and v directions are given by

$$\mathbf{f}_{u,i} = -a k_s C_u \nabla_{\mathbf{x}_i} C_u, \text{ and } \mathbf{f}_{v,i} = -a k_s C_v \nabla_{\mathbf{x}_i} C_v, \quad (1.7)$$

respectively, where the gradient of C_u and C_v with respect to the three triangle vertices are

$$\begin{aligned} \nabla_{\mathbf{x}} C_u &= \begin{bmatrix} -(d_{00} + d_{10}) \frac{\mathbf{w}_u}{\|\mathbf{w}_u\|_2} & d_{00} \frac{\mathbf{w}_u}{\|\mathbf{w}_u\|_2} & d_{10} \frac{\mathbf{w}_u}{\|\mathbf{w}_u\|_2} \end{bmatrix} \\ \nabla_{\mathbf{x}} C_v &= \begin{bmatrix} -(d_{01} + d_{11}) \frac{\mathbf{w}_v}{\|\mathbf{w}_v\|_2} & d_{01} \frac{\mathbf{w}_v}{\|\mathbf{w}_v\|_2} & d_{11} \frac{\mathbf{w}_v}{\|\mathbf{w}_v\|_2} \end{bmatrix}. \end{aligned} \quad (1.8)$$

You'll integrate the cloth using linearized implicit Euler (see details in Sec. 1.3.2). To populate the linear system, we also need to compute the second-order derivatives $\frac{\partial^2 C}{\partial \mathbf{x}_i \partial \mathbf{x}_j}$ (recall your PA2). According to (1.8), we need to derive $\frac{\partial}{\partial \mathbf{x}_i} \frac{\mathbf{w}_u}{\|\mathbf{w}_u\|_2}$ and $\frac{\partial}{\partial \mathbf{x}_i} \frac{\mathbf{w}_v}{\|\mathbf{w}_v\|_2}$ by chain rule. Here are the expressions:

$$\begin{aligned} \nabla_{\mathbf{x}} \frac{\mathbf{w}_u}{\|\mathbf{w}_u\|_2} &= \begin{bmatrix} -\frac{d_{00} + d_{10}}{\|\mathbf{w}_u\|_2} & \frac{d_{00}}{\|\mathbf{w}_u\|_2} & \frac{d_{10}}{\|\mathbf{w}_u\|_2} \end{bmatrix} (\mathbf{Id} - \tilde{\mathbf{w}}_u \tilde{\mathbf{w}}_u^T) \\ \nabla_{\mathbf{x}} \frac{\mathbf{w}_v}{\|\mathbf{w}_v\|_2} &= \begin{bmatrix} -\frac{d_{01} + d_{11}}{\|\mathbf{w}_v\|_2} & \frac{d_{01}}{\|\mathbf{w}_v\|_2} & \frac{d_{11}}{\|\mathbf{w}_v\|_2} \end{bmatrix} (\mathbf{Id} - \tilde{\mathbf{w}}_v \tilde{\mathbf{w}}_v^T), \end{aligned} \quad (1.9)$$

where $\tilde{\mathbf{w}}_u = \frac{\mathbf{w}_u}{\|\mathbf{w}_u\|_2}$ and $\tilde{\mathbf{w}}_v = \frac{\mathbf{w}_v}{\|\mathbf{w}_v\|_2}$ are the normalized vectors of $\mathbf{w}_u$ and $\mathbf{w}_v$, respectively, and $\mathbf{Id} - \tilde{\mathbf{w}}_u \tilde{\mathbf{w}}_u^T$ is known as the projection matrix onto the plane perpendicular to $\tilde{\mathbf{w}}_u$.

Assignment Implementation. In your assignment, you will use (1.7) to compute stretch force in `eval_cloth_stretch_shear_forces(...)` in `forces.py` and *accumulate* the force in `state.particle.f`. Eq. (1.8) will be needed for both computing the force and the second-order derivatives. The latter, together with Eq. (1.9), is needed for populating the linear system in 1.3.2.

Shear Force

The constraint for shear forces is given by $C = \mathbf{w}_u^T \mathbf{w}_v$, and the shear energy is given by

$$E_s = \frac{1}{2} a k_r C^2. \quad (1.10)$$

We can derive the gradient of C with respect to $\mathbf{x}_0$, $\mathbf{x}_1$ and $\mathbf{x}_2$ by chain rule (or using computer), and obtain:

$$\nabla_{\mathbf{x}} C = [-(d_{00} + d_{10})\mathbf{w}_v - (d_{01} + d_{11})\mathbf{w}_u \quad d_{00}\mathbf{w}_v + d_{01}\mathbf{w}_u \quad d_{10}\mathbf{w}_v + d_{11}\mathbf{w}_u]. \quad (1.11)$$

With this gradient, you can compute the shear force

$$\mathbf{f}_{s,i} = -ak_r C \nabla_{\mathbf{x}_i} C \quad (1.12)$$

for each vertex i on that triangle. Again, we need the second-order derivatives $\frac{\partial^2 C}{\partial \mathbf{x}_i \partial \mathbf{x}_j}$, which by chain rule (by differentiating (1.11) w.r.t. $\mathbf{x}_i$) requires $\frac{\partial}{\partial \mathbf{x}_i} \mathbf{w}_u$ and $\frac{\partial}{\partial \mathbf{x}_i} \mathbf{w}_v$, and the expressions are simply:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{x}} \mathbf{w}_u &= [-(d_{00} + d_{10}) \quad d_{00} \quad d_{10}] \mathbf{Id} \\ \frac{\partial}{\partial \mathbf{x}} \mathbf{w}_v &= [-(d_{01} + d_{11}) \quad d_{01} \quad d_{11}] \mathbf{Id}. \end{aligned} \quad (1.13)$$

Assignment Implementation. The expressions here are in parallel to those in the *Stretch Force* section. You will need them when implementing `eval_cloth_stretch_shear_forces(...)` as well.

Bending Force

Let the two adjacent triangles be formed by four vertices:

- $\mathbf{x}_1, \mathbf{x}_2$: The vertices forming the shared “hinge” edge.
- $\mathbf{x}_3, \mathbf{x}_4$: The “tip” vertices opposite the shared edge.

The bending constraint is simply $C = \theta$, where θ is the bending angle between the two triangles (see Figure 1.3). When the two triangles are flat, $\theta = 0$.

The bending energy is given by

$$E_b = \frac{1}{2} k_b C^2.$$

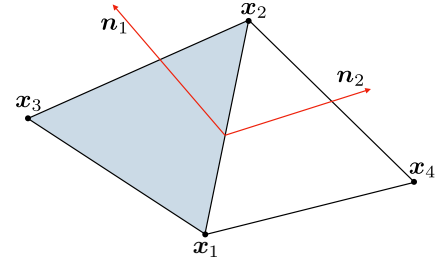


Figure 1.3: Bending force around an edge.

Notice that here the bending energy is *not* scaled by the triangle area, as has been explained in the paper [1] (see footnote in Sec. 4.3).

We now derive the gradient of C with respect to $\mathbf{x}_0$, $\mathbf{x}_1$, $\mathbf{x}_2$, $\mathbf{x}_3$ and $\mathbf{x}_4$.

First, the shared edge by both triangle is $\mathbf{e} = \mathbf{x}_2 - \mathbf{x}_1$, and the two triangle normals (scaled by triangle areas) are $\mathbf{n}_1 = \mathbf{e} \times (\mathbf{x}_3 - \mathbf{x}_1)$ and $\mathbf{n}_2 = (\mathbf{x}_4 - \mathbf{x}_1) \times \mathbf{e}$. They determine the values of $\cos(\theta)$ and $\sin(\theta)$, i.e.,

$$\begin{aligned} \cos(\theta) &= \frac{\mathbf{n}_1}{\|\mathbf{n}_1\|} \cdot \frac{\mathbf{n}_2}{\|\mathbf{n}_2\|} \\ \sin(\theta) &= \left(\frac{\mathbf{n}_1}{\|\mathbf{n}_1\|} \times \frac{\mathbf{n}_2}{\|\mathbf{n}_2\|} \right) \cdot \frac{\mathbf{e}}{\|\mathbf{e}\|}. \end{aligned} \quad (1.14)$$

With these two values, we can compute θ by $\theta = \arctan 2(\sin(\theta), \cos(\theta))$, where `arctan2` is a function provided by `numpy`.

Next, we derive the gradient of θ with respect to $\mathbf{x}_1$, $\mathbf{x}_2$, $\mathbf{x}_3$, and $\mathbf{x}_4$. Let h_1 be the height on $\mathbf{e}$ of the triangle formed by $\mathbf{x}_1$, $\mathbf{x}_2$ and $\mathbf{x}_3$. The triangle area is $\frac{1}{2} \|\mathbf{n}_1\|$, which equals to $\frac{1}{2} h_1 \|\mathbf{e}\|$. Therefore, $h_1 = \|\mathbf{n}_1\| / \|\mathbf{e}\|$.

For a small movement $d\mathbf{x}_3$ along the normal $\mathbf{n}_1$, the change in angle is

$$d\theta = -\frac{d\mathbf{x}_3 \cdot \hat{\mathbf{n}}_1}{h_1}. \quad (1.15)$$

The negative sign here is due to the definition of θ using $\arctan2$ (recall Figure...).

Now, substitute h_1 and the unit normal $\hat{\mathbf{n}}_1 = \frac{\mathbf{n}_1}{\|\mathbf{n}_1\|}$ into the equation:

$$\nabla_{\mathbf{x}_3} \theta = \frac{d\theta}{d\mathbf{x}_3} = -\frac{\hat{\mathbf{n}}_1}{h_1} = -\frac{\|\mathbf{e}\|}{\|\mathbf{n}_1\|^2} \mathbf{n}_1. \quad (1.16)$$

and similarly,

$$\nabla_{\mathbf{x}_4} \theta = -\frac{\hat{\mathbf{n}}_2}{h_2} = -\frac{\|\mathbf{e}\|}{\|\mathbf{n}_2\|^2} \mathbf{n}_2. \quad (1.17)$$

Because moving all vertices by the same amount doesn't change θ , the sum of all internal gradients must be zero, i.e.,

$$\nabla_{\mathbf{x}_1} \theta + \nabla_{\mathbf{x}_2} \theta + \nabla_{\mathbf{x}_3} \theta + \nabla_{\mathbf{x}_4} \theta = 0. \quad (1.18)$$

Together with (1.16) and (1.17), we can derive the other two gradients (I skipped the detailed derivation here, leaving them as an exercise for you).

$$\begin{aligned} \nabla_{\mathbf{x}_1} \theta &= -\frac{(\mathbf{x}_2 - \mathbf{x}_3) \cdot \mathbf{e}}{\|\mathbf{e}\|^2} \nabla_{\mathbf{x}_3} \theta - \frac{(\mathbf{x}_2 - \mathbf{x}_4) \cdot \mathbf{e}}{\|\mathbf{e}\|^2} \nabla_{\mathbf{x}_4} \theta \\ \nabla_{\mathbf{x}_2} \theta &= \frac{(\mathbf{x}_1 - \mathbf{x}_3) \cdot \mathbf{e}}{\|\mathbf{e}\|^2} \nabla_{\mathbf{x}_3} \theta + \frac{(\mathbf{x}_1 - \mathbf{x}_4) \cdot \mathbf{e}}{\|\mathbf{e}\|^2} \nabla_{\mathbf{x}_4} \theta \end{aligned} \quad (1.19)$$

With these expressions, the bending force on $\mathbf{x}_i$ is given by

$$\mathbf{f}_{b,i} = -k_b \theta \nabla_{\mathbf{x}_i} \theta \quad (1.20)$$

for $i = 1, 2, 3, 4$. It is much more involved to derive the second order derivatives $\frac{\partial^2 \theta}{\partial \mathbf{x}_i \partial \mathbf{x}_j}$ of θ with respect to $\mathbf{x}_i$. Luckily, we can avoid using them in the time integrator, as explained in the next section.

1.3.2 IMEX (Implicit-Explicit) Integration

The reason that we have to resort to implicit time integration is that a cloth is largely in-stretchable. But most clothes are easily bendable. In other words, the bending force is much weaker than the stretch and shear forces. Therefore, we can use an explicit time integration scheme for the bending force, and use an implicit time integration scheme for the stretch and shear forces, without losing stability. This is the idea of IMEX (Implicit-Explicit) integration—you get the stability of implicit integration for the stiffest forces (stretch and shear) without the computational nightmare of the bending Hessian.

In particular, when deriving the (linearized) implicit Euler (recall PA2), we split the force into two parts:

$$\mathbf{F}(\mathbf{q}^n + h\dot{\mathbf{q}}^n, \dot{\mathbf{q}}^n) = \mathbf{F}_i(\mathbf{q}^n + h\dot{\mathbf{q}}^n, \dot{\mathbf{q}}^n) + \mathbf{F}_e(\mathbf{q}^n, \dot{\mathbf{q}}^n), \quad (1.21)$$

where $\mathbf{F}_i$ include all stiff forces (namely stretch and share forces), and $\mathbf{F}_e$ include all soft forces (namely bending and contact forces). Notice that we express $\mathbf{F}_e$ with the state at the beginning of a timestep (i.e., in explicit Euler fashion), while expressing $\mathbf{F}_i$ with the position at the end of a timestep (i.e., in implicit Euler fashion). In PA2, you implemented linearized Implicit Euler scheme:

$$\left[\mathbf{M} - \left(h^2 \frac{\partial \mathbf{F}}{\partial \mathbf{q}} + h \frac{\partial \mathbf{F}}{\partial \dot{\mathbf{q}}} \right) \right] \delta \dot{\mathbf{q}} = h \mathbf{F}(\mathbf{q}^n + h\dot{\mathbf{q}}^n, \dot{\mathbf{q}}^n) \quad (1.22)$$

With the force split as in (1.21), we can rewrite the equation as:

$$\left[\mathbf{M} - \left(h^2 \frac{\partial \mathbf{F}_i}{\partial \mathbf{q}} + h \frac{\partial \mathbf{F}_i}{\partial \dot{\mathbf{q}}} \right) \right] \delta \dot{\mathbf{q}} = h \mathbf{F}_i(\mathbf{q}^n + h\dot{\mathbf{q}}^n, \dot{\mathbf{q}}^n) + h \mathbf{F}_e(\mathbf{q}^n, \dot{\mathbf{q}}^n) \quad (1.23)$$

In this way, the bending force only appears on the right hand side, thus no second-order derivatives of the bending constraints are needed. $\mathbf{F}_i$ includes both the stretch and shear forces. Therefore, on the left hand

side, we need $\frac{\partial \mathbf{F}}{\partial \mathbf{q}}$, which is given by Eq. (8) in the paper [1]. **Note that in your implementation, you should replace k in (8) with ak** due to our definition of the energy functions (1.6) and (1.10). The damping force on vertex i has the form

$$\mathbf{F}_{d,i} = -ak_d \frac{\partial C(\mathbf{x})}{\partial \mathbf{x}_i} \dot{C}(\mathbf{x}) = -ak_d \frac{\partial C(\mathbf{x})}{\partial \mathbf{x}_i} \left(\frac{\partial C(\mathbf{x})}{\partial \mathbf{x}} \cdot \dot{\mathbf{x}} \right). \quad (1.24)$$

Please refer to Sec. 4.5 of the paper [1] for detailed explanations. Notice that since we have multiple C functions (for stretch and shear on each triangle), each contributes to a damping force on certain vertices. Damping forces also contribute to the third term $\frac{\partial \mathbf{F}_i}{\partial \mathbf{q}}$ on the left-hand side of (1.23) (see the last equation of Sec. 4.5 in [1]). **Also note that** in Eq. (12) of the paper [1], the first term that involves $\frac{\partial C(\mathbf{x})}{\partial \mathbf{x}_i} \left(\frac{\partial \dot{C}(\mathbf{x})}{\partial \mathbf{x}_j} \right)^T$ can be safely ignored, because it is typically much smaller than the second term, as explained in the paper, that is, only the second term of (12) in the paper is considered when constructing the linear system according to (1.23).

Chapter 2

Assignment Requirements

In this assignment, we will implement a cloth simulation. I already implemented the framework of IMEX integrator in `nemo/integrators/large_step_cloth.py`, and you don't need to touch that file. The linear system constructed in `nemo/solvers/large_step_cloth.py` is given by (1.23), stored as a block sparse matrix (see `nemo/sim/sparse.py` for details). We solve the linear system using preconditioned conjugate gradient method (PCG) provided by `scipy.sparse.linalg`. If you want to learn the details of PCG, I strongly recommend you to read [this course note](#).

We don't consider collisions between the cloth and other objects (including the cloth itself). This is because the collision detection and response is not the focus of this assignment and is very expensive. In fact, the major bottleneck of a cloth simulation is the collision detection and response. We therefore choose not to deal with it in this assignment, and you'll see the cloth intersects with itself.

Your mission is to implement two methods in `nemo/sim/forces.py`: `eval_cloth_stretch_shear_forces(...)` and `eval_cloth_bending_forces(...)`. Please read the comments I left there carefully for hints.

Implementation Suggestions. `eval_cloth_stretch_shear_forces(...)` is a fairly complex function, taking into account all the stretch and shear forces on all triangles and their damping forces. I recommend you to break it down into several steps:

1. Implement only the shear force without damping in `eval_cloth_stretch_shear_forces(...)`. This should already produce reasonable cloth motion (i.e., the cloth won't stretch much and the motions are smooth).
2. Add damping to the stretch force in `eval_cloth_stretch_shear_forces(...)`, according to Sec. 4.5 of [1] and the derivation above.
3. Implement `eval_cloth_bending_forces(...)` to add bending forces and the damping. By then, you should already have a nicely looking cloth simulation.
4. Finally, add the shear forces and the damping into `eval_cloth_stretch_shear_forces(...)`.

Bibliography

- [1] David Baraff and Andrew Witkin. Large steps in cloth simulation. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 767–778. 2023.